Java Concurrency & Multithreading — Study Notes

From threads-from-scratch to the concurrency toolkit actually used in this codebase.

> *Every section that maps to something real in `student-grade-management/src/main/concurrent/` (or `GradeManager`) has a **"Used in this project"** box: the exact file, what it does, and — the important part — why that tool and not a simpler one. Sections with no such box (Fork/Join, reactive streams, `Semaphore`, `CountDownLatch`, deadlock/livelock) aren't used here; they're covered generically so this guide is complete on its own.*

Table of Contents

---

1. Process vs Thread

A **process** is a running program with its own memory space, file handles, and OS-level resources — the JVM itself is one process. A **thread** is a unit of execution *inside* a process; all threads in the same JVM process share the same heap (objects, static fields) but each gets its own **call stack** and

program counter.

	Process	Thread
Memory	Isolated (own address space)	Shared (same heap)
Creation cost	Expensive (OS allocates a new address space)	Cheap (shares the parent's memory)
Communication	Needs IPC (sockets, pipes, shared files)	Direct — shared variables
Crash isolation	One process crashing doesn't kill another	One thread's uncaught exception can, if unhandled, terminate that thread (not the whole JVM, but see below)
Java default	1 process = 1 JVM instance	Every JVM starts with at least a `main` thread, plus GC threads, JIT compiler threads, etc.

Why this matters for concurrency: because threads share memory, every concurrency bug in this guide — race conditions, visibility problems, deadlocks — exists *because* threads share mutable state. Processes don't have these problems (they'd need IPC bugs instead), but they pay for that isolation with much higher creation/communication cost. This is exactly why a single JVM process uses many cheap threads instead of many expensive processes to get parallelism.

An uncaught exception on a non-main thread does **not** crash the JVM — it terminates only that thread and prints a stack trace via the thread's `UncaughtExceptionHandler`. This surprises beginners: a `RuntimeException` thrown inside a background thread can silently disappear if nothing is watching that thread's result.

```
Thread t = new Thread(() -> { throw new RuntimeException("boom"); }); t.start(); //
JVM keeps running. The exception is printed to stderr by the // default uncaught
exception handler, but nothing "bubbles up" to main().
```

---

2. Thread Lifecycle & Creation

The lifecycle

A Java thread moves through six states (`Thread.State` enum):

```
NEW → RUNNABLE → (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED
```

- **NEW** — `Thread` object created, `start()` not yet called.
- **RUNNABLE** — eligible to run; the OS scheduler decides when it actually gets CPU time (Java doesn't distinguish "ready" from "running" as a separate state).
- **BLOCKED** — waiting to acquire a `synchronized` lock someone else holds.

- **WAITING** / **TIMED_WAITING** — parked via `Object.wait()`, `Thread.join()`, `LockSupport.park()`, or a timed sleep.
- **TERMINATED** — `run()` returned or threw.

```
Thread t = new Thread(() -> {}); System.out.println(t.getState()); // NEW t.start();
System.out.println(t.getState()); // RUNNABLE (probably) t.join();
System.out.println(t.getState()); // TERMINATED
```

Two ways to create a thread

1. Extend `Thread` — rarely the right choice; it burns your one shot at extending a class, and couples "what runs" to "how it's threaded."

```
class ReportWorker extends Thread { @Override public void run() {
System.out.println("working..."); } } new ReportWorker().start();
```

2. Implement `Runnable` (preferred) — separates *what to do* from *the mechanism that runs it*, and composes with the entire `java.util.concurrent` toolkit (Section 5), since `ExecutorService.submit()` takes a `Runnable`/`Callable`, not a `Thread` subclass.

```
Runnable task = () -> System.out.println("working..."); new Thread(task).start(); //
still works with a raw Thread... executorService.submit(task); // ...but now also
works with a pool
```

Rule of thumb: never extend `Thread` in application code. Write a `Runnable` (or `Callable<T>` if it needs to return a value — Section 5) and hand it to a `Thread` or, almost always, to an `ExecutorService`.

`Runnable` vs `Callable<T>`

```
Runnable r = () -> System.out.println("no return value, can't throw checked
exceptions"); Callable<Integer> c = () -> { if (Math.random() > 0.5) throw new
Exception("can throw checked exceptions"); return 42; // has a return value };
```

`run()` returns `void` and can't declare checked exceptions; `call()` returns `V` and can throw `Exception`. Use `Callable` whenever the background work produces a result or can legitimately fail with a checked exception — see `Future`/`ExecutorService` in Section 5.

---

3. Thread Management

`start()` vs calling `run()` directly

This is the single most common beginner mistake:

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));
t.run(); // WRONG: runs on the CURRENT thread, synchronously, no new thread at all
t.start(); // RIGHT: schedules run() to execute on a NEW thread
```

Calling `.run()` is just an ordinary method call — no concurrency happens. Only `.start()` asks the JVM/OS to create a new thread of execution.

`join()` — waiting for a thread to finish

```
Thread worker = new Thread(() -> doExpensiveWork()); worker.start(); worker.join(); //
blocks the CALLING thread until worker finishes System.out.println("worker is done,
safe to read its results now");
```

`join()` also has a timeout overload (`join(5000)` — wait at most 5 seconds), which is exactly how this project stops a background thread cleanly — see `StatisticsDashboard.stop()` below.

> ***Used in this project — `StatisticsDashboard.stop()`***
> *(`main/concurrent/StatisticsDashboard.java`)* `java public boolean stop() { if (!running.compareAndSet(true, false)) return false; ticker.interrupt(); try { ticker.join(TimeUnit.SECONDS.toMillis(5)); // wait up to 5s for the ticker thread to exit } catch (InterruptedException e) { Thread.currentThread().interrupt(); } shutdownExecutor(); return true; }`
> ***Why:*** *`stop()` needs to guarantee the ticker thread has actually exited its loop before returning — otherwise a caller could call `stop()` then immediately `start()` again and race with the old ticker still shutting down. `interrupt()` breaks the ticker out of its `Thread.sleep(...)`, and the bounded `join(5000)` prevents `stop()` itself from hanging forever if something goes wrong, instead of an unbounded `join()`.*

Daemon vs non-daemon threads

The JVM exits when **every non-daemon thread has finished** — daemon threads are killed abruptly, mid-execution, the moment that happens. Background/support threads (loggers, schedulers, caches) should almost always be daemon threads, so they never prevent the application from shutting down.

```
Thread background = new Thread(() -> { /* housekeeping */ });
background.setDaemon(true); // must be set BEFORE start() background.start();
```

> ***Used in this project — every background thread here is explicitly daemon.***
> *`AuditTrail.active()`, `ScheduledGpaRecalculationJob.start()`, and `StatisticsDashboard.start()` all name and daemonize their thread the same way: `java ExecutorService executor = Executors.newSingleThreadExecutor(runnable -> { Thread thread = new Thread(runnable, "audit-trail-writer"); // named, for jstack/debugging thread.setDaemon(true); // never blocks JVM shutdown return thread; });` **Why:** an audit-trail writer or a statistics ticker is support infrastructure, not the actual program. If the user picks "Exit" from the console menu, the JVM must be able to shut down immediately — it shouldn't hang because a background audit-log writer thread is still alive.*

> *Naming the thread (`"audit-trail-writer"`, `"dashboard-ticker"`, `"gpa-recalculation-scheduler"`) is a small but real practice: a thread dump (`jstack`) full of `Thread-0`, `Thread-1`, `Thread-2` is useless for debugging; named threads tell you instantly what's running.*

Thread priorities

```
thread.setPriority(Thread.MAX_PRIORITY); // 10
thread.setPriority(Thread.NORM_PRIORITY); // 5, the default
thread.setPriority(Thread.MIN_PRIORITY); // 1
```

Pitfall: thread priority is only a *hint* to the OS scheduler, and its effect is platform-dependent — some OSes honor it closely, others largely ignore it. Never use priority as a correctness mechanism (e.g., "the writer thread has higher priority so it will always run first") — that's a race condition wearing a disguise. Priority is a performance tuning knob at best, not used anywhere in this codebase, and rarely worth reaching for in application code at all.

---

4. Synchronization

The problem: race conditions

A **race condition** happens when two threads read-modify-write shared state without coordination, and the final result depends on unpredictable timing.

```
class Counter { private int count = 0; void increment() { count++; } // NOT atomic!
read, add 1, write — 3 steps }
```

`count++` is three separate bytecode operations. If two threads interleave between the read and the write, one increment can be silently lost:

```
Thread A reads count = 5 Thread B reads count = 5 Thread A writes count = 6 Thread B
writes count = 6 <-- lost update! should have been 7
```

Run 1000 threads each incrementing this counter once and you'll reliably get a final value **less than 1000** — that's a race condition, not a hypothetical.

`synchronized` — intrinsic locks

Every Java object has an implicit **monitor lock**. `synchronized` acquires it for the duration of a block or method, so only one thread can be inside at a time.

```
class Counter { private int count = 0; synchronized void increment() { count++; } //
locks 'this' synchronized int get() { return count; } // reads must ALSO be
synchronized — } // otherwise another thread can see a stale value
```

```
private final Object lock = new Object(); void increment() { synchronized (lock) { // synchronizing on a dedicated private lock object count++; // is safer than synchronizing on 'this' — nothing external } // can accidentally lock on your object and cause contention }
```

Pitfall — synchronizing on the wrong thing. Synchronizing on a mutable field, a `String` literal, or a boxed `Integer` is a classic bug: literals/interned/cached values can be silently shared across unrelated code, causing lock contention (or worse, false "mutual exclusion" between things that were never meant to be mutually exclusive) that has nothing to do with your actual critical section.

Visibility: `volatile`

`synchronized` gives you both **mutual exclusion** (one thread at a time) and **visibility** (changes made by one thread are guaranteed visible to the next thread that acquires the same lock). Sometimes you only need visibility, with no compound read-modify-write to protect — that's what `volatile` is for:

```
private volatile boolean running = false; // a flag, only ever fully replaced, never "running++" void stop() { running = false; } void loop() { while (running) { /* ... */ } }
```

Without `volatile`, a thread's local CPU cache could keep serving a stale cached `true` for `running` forever, even after another thread sets it to `false` — `loop()` would never see the update and would spin forever. `volatile` forces every read/write to go through main memory (establishes a happens-before relationship), but **it does not make compound operations atomic** — `volatile int count; count++;` is still a race condition, because `++` is still three steps.

> ***Used in this project — `ScheduledGpaRecalculationJob`'s status fields*** *(`main/concurrent/ScheduledGpaRecalculationJob.java`)* `java private volatile Instant lastRunAt; private volatile Instant nextRunAt; private volatile NavigableMap<Double, List<Student>> lastResult;` ***Why `volatile` and not `synchronized`:*** *each field is replaced wholesale by the scheduler thread (`runOnce()` builds a brand-new `Instant`/`NavigableMap` and assigns it — it never mutates an existing one in place), and read by any other thread calling `getLastResult()`. That's exactly the "publish-once, replace-the-whole-reference" pattern `volatile` is built for: the reader either sees the old complete object or the new complete one, never a half-built one, with no lock needed on the read side. If `runOnce()` ever mutated the existing map in place instead of replacing it, `volatile` alone would **not** be enough — see `ReentrantReadWriteLock` in Section 7 for the pattern used when a read needs to be protected against a concurrent write.*

`wait()` / `notify()` / `notifyAll()`

Low-level coordination primitives for one thread to pause until another signals it — the mechanism `BlockingQueue` (Section 12) is built on top of internally, and rarely written by hand in modern code, but essential to recognize:

```
class Buffer { private final Object lock = new Object(); private String data; void produce(String value) { synchronized (lock) { data = value; lock.notify(); // wake up
```

```
one thread waiting on this lock } } String consume() throws InterruptedException {
synchronized (lock) { while (data == null) { lock.wait(); // release the lock and
sleep until notified } String result = data; data = null; return result; } } }
```

Pitfall: always call `wait()` inside a `while` loop re-checking the condition, never an `if` — a *spurious wakeup* (the JVM is allowed to wake a waiting thread without anyone calling `notify()`) or another consumer grabbing the data first would otherwise let a thread proceed on a false premise.

---

5. Concurrency Utilities

Writing raw `Thread`/`synchronized` code for anything beyond a toy example is error-prone and doesn't scale. `java.util.concurrent` (added in Java 5) is the toolkit almost all real code should use instead.

`ExecutorService` — thread pools

An `ExecutorService` decouples *submitting work* from *how threads run it*. You submit `Runnable`/`Callable` tasks; the executor owns a pool of worker threads and reuses them.

```
ExecutorService pool = Executors.newFixedThreadPool(4); pool.submit(() ->
System.out.println("task 1")); pool.submit(() -> System.out.println("task 2"));
pool.shutdown(); // stop accepting new tasks; existing ones finish
```

Common factory methods (all backed by `ThreadPoolExecutor` under the hood):

Factory	Pool shape	Good for
`newFixedThreadPool(n)`	Exactly `n` threads, unbounded queue	Predictable, CPU/IO-bounded parallel work (this project's `BatchReportService`)
`newSingleThreadExecutor()`	1 thread	Serializing work so it never interleaves (this project's `AuditTrail`)
`newCachedThreadPool()`	Grows unbounded, reuses idle threads, shrinks after 60s idle	Many short-lived, bursty tasks (this project's `StatisticsDashboard`)
`newScheduledThreadPool(n)` / `newSingleThreadScheduledExecutor()`	Fixed pool + delay/repeat scheduling	Recurring jobs (this project's `ScheduledGpaRecalculationJob`)

> ***Used in this project — four different executor shapes, deliberately different tools for different jobs:*** *- `AuditTrail.active()` → `Executors.newSingleThreadExecutor(...)` —* ***why single-thread:*** *every audit entry must be written whole, one at a time, in submission order, so concurrent callers can never interleave or corrupt an entry. A single-thread executor makes that a structural guarantee instead of something*

> *you have to remember to lock. - `BatchReportService` → `Executors.newFixedThreadPool(threadCount)` (`threadCount` between `MIN_THREADS=2` and `MAX_THREADS=8`) — **why fixed:** one report-generation task per student is CPU/IO-bound and roughly uniform in cost; a bounded pool gives predictable, measurable parallelism (the class literally measures and reports `elapsedMillis` and `threadPoolSize` to demonstrate the speedup). - `StatisticsDashboard.start()` → `new ThreadPoolExecutor(0, Integer.MAX_VALUE, 60L, SECONDS, new SynchronousQueue<>())` (i.e. a hand-built `newCachedThreadPool()`) — **why cached:** dashboard refreshes are bursty (one every 5 seconds) and cheap; a cached pool spins up a thread only when a refresh is actually running and lets it die when idle, rather than paying for idle fixed-pool threads between ticks. - `ScheduledGpaRecalculationJob.start()` → `Executors.newSingleThreadScheduledExecutor(...)` — **why scheduled + single-thread:** this is one task on a fixed daily schedule; only one run should ever be in flight, so there's no reason for more than one thread, and `scheduleAtFixedRate` handles the timing instead of a hand-rolled sleep loop.*

`Future` and `Callable`

`Callable<V>` is a task that returns a value (and can throw a checked `Exception`); submitting one to an `ExecutorService` gives you back a `Future<V>` — a handle to a result that doesn't exist yet.

```
ExecutorService pool = Executors.newFixedThreadPool(2); Future<Integer> future =
pool.submit(() -> { Thread.sleep(1000); return 42; }); // do other work here while the
task runs in the background... Integer result = future.get(); // BLOCKS until the task
finishes, then returns 42 Integer withTimeout = future.get(2, TimeUnit.SECONDS); //
throws TimeoutException if too slow
```

`Future.get()` re-throws the task's exception wrapped in an `ExecutionException` — you must unwrap `getCause()` to see the real failure.

> ***Used in this project — `BatchReportService.generateBatch()`***
> *(`main/concurrent/BatchReportService.java`) `java List<Future<StudentReportOutcome>> futures = new ArrayList<>(studentIds.size()); for (String studentId : studentIds) { futures.add(executor.submit(() -> generateOne(studentId, kind, filenamePrefix))); } List<StudentReportOutcome> outcomes = new ArrayList<>(futures.size()); for (Future<StudentReportOutcome> future : futures) { outcomes.add(awaitOutcome(future)); // future.get(), with InterruptedException/ExecutionException handling }` **Why:** every student's report is submitted as an independent task up front (so all N tasks start running across the pool immediately), and then the results are collected one at a time — this is the standard "fan out, then fan in" shape. Note it does **not** use `invokeAll()` (which would also work) — the explicit `List<Future<>>` is used here because `generateOne()` never throws outward (it catches its own `RuntimeException` and returns a `StudentReportOutcome.failure(...)`), so one student's failure never poisons the batch — this is the same "skip and continue" philosophy as `CSVParser`/`BulkImportService` (see the sibling `ADVANCED_TOPICS_GUIDE.md`), just applied under real parallelism instead of a sequential loop.*

`CountDownLatch` — wait for N things to happen once

A one-shot gate: a latch starts at count `N`; any thread calling `await()` blocks until `N` `countDown()` calls have happened, from anywhere. It **cannot be reset**.

```
CountDownLatch startSignal = new CountDownLatch(1); CountDownLatch doneSignal = new CountDownLatch(3); for (int i = 0; i < 3; i++) { new Thread(() -> { try { startSignal.await(); // all 3 workers wait for the same starting gun doWork(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); } finally { doneSignal.countDown(); // signal "I'm done" } }).start(); } startSignal.countDown(); // fire the starting gun — release all 3 workers at once doneSignal.await(); // main thread waits until all 3 have finished System.out.println("all workers finished");
```

Classic uses: making N worker threads start at exactly the same moment (for a benchmark), or having a main thread wait for N parallel initialization tasks to complete before proceeding.

`Semaphore` — limit concurrent access to a resource

A counting lock: `N` permits are available; `acquire()` blocks if none are free, `release()` returns one. Unlike a mutex (`synchronized`, 1 permit), a semaphore can allow **more than one** thread through at once.

```
Semaphore connectionLimiter = new Semaphore(3); // e.g. at most 3 concurrent DB connections void queryDatabase() throws InterruptedException { connectionLimiter.acquire(); try { // at most 3 threads are ever inside this block at once runQuery(); } finally { connectionLimiter.release(); // ALWAYS release in a finally block } }
```

Pitfall: forgetting `release()` in a `finally` block permanently leaks a permit — over time every permit leaks and the resource becomes completely unusable, with no exception ever telling you why.

Neither `CountDownLatch` nor `Semaphore` is used in this codebase (its concurrency needs are met by the executor patterns above), but both are essential, commonly-interviewed `java.util.concurrent` primitives.

---

6. Concurrent Collections

Plain `HashMap`, `ArrayList`, etc. are **not thread-safe** — concurrent modification can corrupt their internal structure (not just give a wrong answer, but genuinely infinite-loop or throw `ConcurrentModificationException`). `java.util.concurrent` provides collections designed for concurrent access.

`ConcurrentHashMap`

A hash map that allows concurrent reads and writes without external locking, by internally partitioning its data so unrelated keys don't contend with each other. It never throws `ConcurrentModificationException` during iteration (iteration is *weakly consistent*: it reflects the

state at some point during the iteration, not a frozen snapshot, and never fails).

```
ConcurrentHashMap<String, Integer> scores = new ConcurrentHashMap<>();
scores.put("STU001", 85); scores.computeIfAbsent("STU002", k -> 0); // atomic "insert
if missing" scores.merge("STU001", 5, Integer::sum); // atomic "update if present,
else insert"
```

> ***Used in this project — `LruCache<K, V>`*** *(`main/concurrent/LruCache.java`)* *`java private final ConcurrentHashMap<K, Entry<V>> store = new ConcurrentHashMap<>();`* ***Why `ConcurrentHashMap` and not a synchronized `LinkedHashMap`*** *(the textbook LRU-cache idiom): the class Javadoc says it directly — "every `get/put/invalidate` is lock-free on the map itself." A `synchronized LinkedHashMap` would serialize every cache access behind one lock, even two threads reading two completely unrelated keys. `ConcurrentHashMap` lets unrelated keys proceed genuinely in parallel; recency for LRU eviction is tracked separately via a shared `AtomicLong` clock stamped onto each entry (Section 7), so eviction never needs its own lock either — it just scans the entry set (safe to do concurrently on a `ConcurrentHashMap`) for the oldest stamp.*

`CopyOnWriteArrayList`

A `List` where every mutation (`add`, `remove`, ...) copies the entire underlying array. Reads/iteration never need a lock and never see a `ConcurrentModificationException`, because they're always working against an immutable snapshot array. This trade-off is only good when **reads vastly outnumber writes** — every write is `O(n)` because the whole array is copied.

```
CopyOnWriteArrayList<String> log = new CopyOnWriteArrayList<>(); log.add("event 1");
for (String entry : log) { // safe to iterate even if another thread calls log.add()
right now — System.out.println(entry); // this loop is iterating a snapshot taken when
the loop started }
```

> ***Used in this project — `AuditTrail`'s entry list*** *(`main/concurrent/AuditTrail.java`)* *`java private final List<AuditEntry> entries; // built as new CopyOnWriteArrayList<>() public List<AuditEntry> getEntries() { return List.copyOf(entries); }`* ***Why:*** *writes to the audit trail already go through a single-thread executor (Section 5), so there's never write-write contention to worry about — but `getEntries()` can be called from any thread at any time to inspect a point-in-time snapshot while the writer thread might be mid-append. `CopyOnWriteArrayList` makes that read completely safe and lock-free without needing to coordinate with the writer at all, which fits the access pattern (rare writes relative to the program's lifetime, occasional reads) exactly.*

`BlockingQueue`

A queue where `put()` blocks if the queue is full (for bounded queues) and `take()` blocks if it's empty — the standard building block for producer-consumer pipelines (Section 12).

```
BlockingQueue<String> queue = new LinkedBlockingQueue<>(10); // capacity 10
queue.put("item"); // blocks if queue is full String item = queue.take(); // blocks if
queue is empty
```

Not used directly in this codebase, but `ThreadPoolExecutor` uses one internally to hold pending tasks (a `SynchronousQueue` in `StatisticsDashboard`'s hand-built cached pool — a queue with *zero* capacity, where every `put()` must rendezvous directly with a `take()`, which is exactly what makes `newCachedThreadPool()`'s "spin up a new thread if none are idle" behavior work).

---

7. Advanced Synchronization

`ReentrantLock` — an explicit alternative to `synchronized`

Everything `synchronized` does, but as an object you can hold a reference to, with extra capabilities: `tryLock()` (don't block forever), timed acquisition, and **fairness** (FIFO ordering among waiting threads).

```
private final ReentrantLock lock = new ReentrantLock(); void criticalSection() {
lock.lock(); try { // ... protected work ... } finally { lock.unlock(); // MUST be in
a finally block — unlike synchronized, nothing unlocks it for you } }
```

Pitfall: unlike `synchronized`, which releases its lock automatically even if an exception is thrown, `ReentrantLock.unlock()` must be called explicitly — always in a `finally` block, or a thrown exception leaves the lock held forever.

`ReadWriteLock` / `ReentrantReadWriteLock`

Splits one lock into a **read lock** (shared — many readers can hold it simultaneously) and a **write lock** (exclusive — one writer at a time, and no readers while a writer holds it). This is a genuine improvement over `synchronized`/plain `ReentrantLock` for **read-heavy** workloads: N threads all just reading don't block each other at all.

```
private final ReadWriteLock lock = new ReentrantReadWriteLock(); void write(int value)
{ lock.writeLock().lock(); try { data = value; } finally { lock.writeLock().unlock();
} } int read() { lock.readLock().lock(); try { return data; } finally {
lock.readLock().unlock(); } }
```

> ***Used in this project — `GradeManager`'s shared lock*** *(`main/manager/GradeManager.java`)*
> *```java private final ReadWriteLock lock = new ReentrantReadWriteLock();*

> *public void addGrade(Grade grade) { lock.writeLock().lock(); try { gradeService.recordGrade(grade); gradeCache.invalidate(grade.getStudentId()); auditTrail.append(...); } finally { lock.writeLock().unlock(); } }*

> *public T readLocked(Supplier read) { lock.readLock().lock(); try { return read.get(); } finally { lock.readLock().unlock(); }} `` **Why:**StatisticsDashboard(every 5 seconds) andScheduledGpaRecalculationJob(daily) both need to read a *consistent* snapshot of grade data — one that can never be half-written by a concurrentaddGrade()call from the console. Both callgradeManager.readLocked(...)to do their reads. Because reads (dashboard refreshes, GPA recalculation) are far more frequent than writes (a teacher recording one grade at a time), a plainsynchronized/ReentrantLockwould force every dashboard tick and every grade-entry to serialize behind each other even when there's no real conflict. The read/write split lets the dashboard tick and the GPA job both read concurrently with each other; only an actualaddGrade()` needs to briefly become exclusive.*

Atomic variables (`java.util.concurrent.atomic`)

Lock-free, thread-safe single-variable operations built on the CPU's **compare-and-swap (CAS)** instruction: "set this value to X, but only if it's still currently Y" — atomically, in hardware, no lock needed.

```
AtomicInteger counter = new AtomicInteger(0); counter.incrementAndGet(); // atomic ++, no race condition possible counter.compareAndSet(5, 10); // "if I'm still 5, become 10" — returns false if someone beat you to it AtomicBoolean flag = new AtomicBoolean(false); if (flag.compareAndSet(false, true)) { // classic "only one thread wins this race" idiom System.out.println("I'm the one thread that gets here"); }
```

Atomics are almost always **faster** than locks for single-variable updates under contention, because CAS never blocks a thread — a failed CAS just means "try again," not "go to sleep and wait to be woken up."

> ***Used in this project — two different atomics, two different jobs:*** *- `StatisticsDashboard.running`: `AtomicBoolean`, used purely for its `compareAndSet` — `start()` does `running.compareAndSet(false, true)` and returns `false` if the dashboard was already running, and `stop()` mirrors it with `compareAndSet(true, false)`.* ***Why:*** *this is the textbook "only one thread wins the race to start/stop" idiom — two threads calling `start()` at the exact same moment must have exactly one of them actually start the ticker thread, and `compareAndSet` makes that guarantee atomic without any separate lock. - `LruCache`'s `clock`, `hits`, and `misses`: `AtomicLong`, used for their `incrementAndGet()` — every `get()`/`put()` bumps a shared logical clock (for LRU recency ordering) and a hit/miss counter, from potentially many threads at once, with no lock on the cache itself (Section 6).* ***Why:*** *these are simple counters with no compound logic beyond "add one" — exactly the case atomics are fastest at, and using them here is what lets `LruCache` stay fully lock-free end to end (`ConcurrentHashMap` + `AtomicLong`, no `synchronized` anywhere in the class).*

---

8. Fork/Join Framework & Parallel Streams

The **fork/join framework** (`java.util.concurrent.ForkJoinPool`) is designed for **divide-and-conquer** work: recursively split a big task into smaller subtasks, run them in parallel, then combine the results. Its key trick is **work-stealing** — an idle worker thread "steals" queued subtasks from a busy worker's queue instead of sitting idle, which keeps all CPU cores busy even when the subtasks are unevenly sized.

```
class SumTask extends RecursiveTask<Long> { private final long[] array; private final int start, end; private static final int THRESHOLD = 1000; SumTask(long[] array, int start, int end) { this.array = array; this.start = start; this.end = end; } @Override protected Long compute() { if (end - start <= THRESHOLD) { long sum = 0; for (int i = start; i < end; i++) sum += array[i]; return sum; // base case: just compute directly } int mid = (start + end) / 2; SumTask left = new SumTask(array, start, mid); SumTask right = new SumTask(array, mid, end); left.fork(); // run 'left' asynchronously on another worker thread long rightResult = right.compute(); // compute 'right' on THIS thread long leftResult = left.join(); // wait for 'left', combine results return leftResult + rightResult; } } long total = ForkJoinPool.commonPool().invoke(new SumTask(bigArray, 0, bigArray.length));
```

Parallel streams — Fork/Join without writing Fork/Join code

Every parallel stream runs on the JVM-wide **common `ForkJoinPool`** under the hood:

```
List<Grade> grades = ...; double average = grades.parallelStream()
.mapToDouble(Grade::getGrade) .average() .orElse(0.0);
```

When parallel streams help: large collections (thousands+ elements), CPU-bound per-element work, and a source that splits evenly (arrays, `ArrayList` — good; `LinkedList`, I/O-bound streams — bad, because they can't be split efficiently or they'll just block the shared pool).

Pitfall — this project deliberately avoids `parallelStream()`. All of this codebase's collection sizes are tiny (max 50 students, max 200 grades — see `CLAUDE.md`), and a parallel stream's overhead (splitting the source, coordinating subtasks, merging results) would be pure waste, possibly even slower than a sequential stream. `BatchReportService`'s explicit `ExecutorService` (Section 5) was chosen instead for a more important reason too: it needed **per-task failure isolation** (one student's failure shouldn't corrupt the batch) and **a dedicated, sized, disposable thread pool** it fully controls the lifecycle of — a parallel stream's shared common pool offers neither of those; an unhandled exception inside a parallel stream operation aborts the whole stream instead of letting you collect a `StudentReportOutcome.failure(...)` per element.

Pitfall — never block inside a parallel stream (e.g. calling `Thread.sleep()` or blocking I/O per element). Because parallel streams share the *one* common `ForkJoinPool` the entire JVM uses (including for other libraries), blocking one of its limited worker threads can starve unrelated parallel work happening elsewhere in the same application.

---

9. CompletableFuture & Async Programming

`CompletableFuture<T>` is `Future<T>` with composition: instead of blocking on `get()`, you chain what should happen *when* the result arrives, without ever blocking a thread to wait for it.

```
CompletableFuture<Integer> future = CompletableFuture .supplyAsync(() ->
fetchScoreFromSlowService()) // runs on the common ForkJoinPool by default
.thenApply(score -> score * 2) // transform the result when it arrives
.thenApply(doubled -> "Score: " + doubled) .exceptionally(ex -> "failed: " +
ex.getMessage()); // handle failure instead of throwing
future.thenAccept(System.out::println); // fire-and-forget once it completes — no
blocking anywhere
```

Combining multiple independent async operations:

```
CompletableFuture<Integer> mathAvg = CompletableFuture.supplyAsync(() ->
averageFor("MATH01")); CompletableFuture<Integer> engAvg =
CompletableFuture.supplyAsync(() -> averageFor("ENGL01")); CompletableFuture<Integer>
combined = mathAvg.thenCombine(engAvg, (m, e) -> (m + e) / 2);
CompletableFuture.allOf(mathAvg, engAvg).join(); // wait for ALL of a collection of
futures
```

Run the async stage on a specific executor instead of the shared common pool (usually the right call in a real application, so you don't compete with unrelated `parallelStream()` work for the same pool):

```
CompletableFuture.supplyAsync(() -> doWork(), myExecutorService);
```

> ***Used in this project (the simple case) — `AuditTrail.append()`'s no-op path*** *(`main/concurrent/AuditTrail.java`) `java public Future<Boolean> append(String action, String entityType, String entityId, String details) { if (executor == null) { return CompletableFuture.completedFuture(null); // AuditTrail.noOp() case } AuditEntry entry = new AuditEntry(action, entityType, entityId, details, Instant.now()); return executor.submit(() -> entries.add(entry)); }* ***Why `CompletableFuture.completedFuture(null)` here, specifically:*** *`AuditTrail.noOp()` exists so every pre-existing caller of `StudentManager`/`GradeManager` (essentially the whole test suite, per the class's own Javadoc) keeps compiling and passing unmodified, without ever creating a real thread. Both branches of `append()` must return the same type (`Future<Boolean>`), so the no-op path needs a `Future` — and `CompletableFuture.completedFuture(...)` is the standard, zero-thread way to hand back "a future that's already done" without spinning up an executor just to immediately resolve one value. This project doesn't chain `.thenApply()`/`.thenCombine()` anywhere (its async needs are simple "submit and maybe wait" via `Future`, Section 5), but this line is exactly where `CompletableFuture` earns its place over a hand-written `Future` implementation: nobody wants to write a custom `Future<Boolean>` class just to represent "already done, value doesn't matter."*

---

10. Reactive Programming Basics

Reactive programming models data as an asynchronous **stream of events over time** — instead of pulling one result once (`Future.get()`), you subscribe to a stream and react to each element as it arrives, with built-in **backpressure** (the consumer can tell the producer to slow down instead of being overwhelmed).

Java 9 added the minimal `java.util.concurrent.Flow` API — three interfaces that popular libraries (Project Reactor, RxJava, Akka Streams) implement or interoperate with:

```
interface Publisher<T> { void subscribe(Subscriber<? super T> subscriber); } interface Subscriber<T> { void onSubscribe(Subscription s); void onNext(T item); void onError(Throwable t); void onComplete(); } interface Subscription { void request(long n); void cancel(); }
```

A minimal example using the JDK's own `SubmissionPublisher` (a ready-made `Flow.Publisher`):

```
SubmissionPublisher<String> publisher = new SubmissionPublisher<>(); publisher.subscribe(new Flow.Subscriber<String>() { private Flow.Subscription subscription; public void onSubscribe(Flow.Subscription s) { subscription = s; subscription.request(1); } public void onNext(String item) { System.out.println("received: " + item); subscription.request(1); } public void onError(Throwable t) { t.printStackTrace(); } public void onComplete() { System.out.println("done"); } }); publisher.submit("event 1"); publisher.submit("event 2"); publisher.close();
```

When reactive programming earns its complexity: high-throughput event streams where a slow consumer must be able to push back on a fast producer (e.g. a service reading a message queue faster than a database can absorb writes) — the backpressure protocol is the entire point. **When it doesn't:** small, bounded, synchronous-ish workloads like this console application, where a simple `Future`/`ExecutorService` (Section 5) is far easier to read, test, and reason about. This codebase has no reactive code, and shouldn't — there's no unbounded event stream or backpressure problem anywhere in a 50-student, 200-grade in-memory console app. Recognizing *when a tool doesn't fit the problem* is as much a part of this skill as knowing the API.

---

11. Common Problems: Deadlock, Starvation, Livelock

Deadlock

Two (or more) threads each hold a lock the other needs, and neither will ever release theirs — both wait forever.

```
// Thread 1: synchronized(lockA) { synchronized(lockB) { ... } } // Thread 2: synchronized(lockB) { synchronized(lockA) { ... } } // If Thread 1 grabs lockA and Thread 2 grabs lockB at the same moment, both are now // stuck forever waiting for the lock the OTHER one is holding.
```

The classic fix: consistent lock ordering. Every thread that needs both locks must always acquire them in the *same* order (e.g. always `lockA` before `lockB`, everywhere in the codebase) — this makes

the circular-wait condition above structurally impossible.

```
// Both threads now acquire in the SAME order: lockA, then lockB. No deadlock
possible. synchronized (lockA) { synchronized (lockB) { // ... } }
```

Other fixes: use `ReentrantLock.tryLock(timeout)` to fail fast instead of blocking forever, or (best of all) avoid needing two locks held simultaneously in the first place — this codebase's `GradeManager` uses exactly **one** lock (`ReentrantReadWriteLock`, Section 7) for its entire critical section, which is why deadlock isn't a risk there: you cannot deadlock on a single lock.

Starvation

A thread is perpetually denied the resources it needs to proceed — not stuck forever like deadlock, just unfairly, indefinitely delayed — usually because other threads keep "cutting in line." Common cause: an unfair lock combined with some threads submitting work far more aggressively than others.

```
ReentrantLock fairLock = new ReentrantLock(true); // fair=true: FIFO queue, no thread
can be perpetually skipped
```

Fair locks trade some throughput for this guarantee — enabling fairness on every lock "just in case" is itself an anti-pattern; use it only when you've actually observed or reasoned about a starvation risk.

Livelock

Threads are actively doing work (not blocked, unlike deadlock) but never make real progress — like two people repeatedly stepping the same direction trying to let each other pass in a hallway.

```
// Two threads, each politely "backing off" when they detect contention, // but
backing off in a way that keeps re-creating the same contention forever: while
(!tryAcquireBothLocks()) { releaseAnyHeldLocks(); Thread.sleep(RANDOM_BACKOFF); // if
both threads back off and retry in lockstep, this can livelock }
```

Fix: make the backoff genuinely randomized/jittered (not a fixed delay both threads share), or — again — avoid needing to coordinate two locks acquired independently at all.

None of these three appear in this codebase's design, and that's not an accident: every concurrent class here (Section 5) is built around **at most one lock or one single-purpose executor**, specifically to avoid ever needing the kind of multi-lock coordination where deadlock/livelock become possible.

---

12. Thread-Safe Design Patterns

Producer-Consumer

One or more **producer** threads generate work items; one or more **consumer** threads process them, coordinated through a `BlockingQueue` (Section 6) so producers block if consumers fall behind, and consumers block (instead of busy-waiting) when there's nothing to do.

```
BlockingQueue<Grade> pendingGrades = new LinkedBlockingQueue<>(100); // Producer new
Thread(() -> { while (true) { Grade grade = readNextGradeFromImportFile();
pendingGrades.put(grade); // blocks if the queue is full (backpressure) } }).start();
// Consumer new Thread(() -> { while (true) { Grade grade = pendingGrades.take(); //
blocks if the queue is empty persistGrade(grade); } }).start();
```

Worker Pool (a.k.a. Thread Pool pattern)

A fixed set of worker threads pull tasks from a shared queue and process them — which is exactly what `ExecutorService` (Section 5) *is*, under the hood, as a reusable abstraction instead of something you hand-roll.

> ***Used in this project — `BatchReportService` is a textbook Worker Pool.*** *A fixed-size pool (2–8 threads, `Executors.newFixedThreadPool`) is handed one task per student; each worker pulls the next queued task as soon as it finishes the previous one, so the total wall-clock time for N students is roughly `N / threadCount` instead of `N` sequential report generations. This is the entire reason the class exists — `main/console/ExportGradeReportAction` already generates a report for one student sequentially; `BatchReportService` is the worker-pool version of the same operation for many students at once, and its own `BatchResult.elapsedMillis` + `threadPoolSize` fields exist specifically so a test can measure and assert the parallel speedup is real, not assumed.*

Single-Writer / Publish-Once

Not one of the "big three" named patterns, but worth naming because this codebase uses it twice: exactly one thread (or one single-thread executor) is ever allowed to *write*, and readers coordinate via `volatile`/`ConcurrentHashMap`/`CopyOnWriteArrayList` rather than locking against the writer. `AuditTrail` (Section 6, single-thread executor + `CopyOnWriteArrayList`) and `ScheduledGpaRecalculationJob` (Section 4, single scheduled thread + `volatile` publish-once fields) are both this pattern — it's a simpler, cheaper alternative to a full read/write lock (Section 7) whenever there's naturally only ever one writer to begin with.

---

13. Best Practices for Writing & Testing Concurrent Code

Always shut down an `ExecutorService` — and do it defensively

Every executor-owning class in this project follows the **exact same three-step shutdown idiom**, because getting this wrong leaks threads that quietly keep the JVM alive forever:

```
executor.shutdown(); // 1. stop accepting new tasks try { if
(!executor.awaitTermination(30, TimeUnit.SECONDS)) { // 2. wait for in-flight tasks to
finish executor.shutdownNow(); // ...but don't wait forever — force-cancel if stuck }
} catch (InterruptedException e) { executor.shutdownNow();
Thread.currentThread().interrupt(); // 3. NEVER swallow an interrupt — restore the
flag }
```

Why each line matters: - `shutdown()` alone lets already-submitted tasks finish but rejects new ones — it does **not** stop the executor or its threads immediately, so it must be paired with an await. - `awaitTermination` with a timeout means a stuck/hung task can't hang your shutdown path forever — a bare, un-timed wait for "all tasks done" is a latent bug. - `Thread.currentThread().interrupt()` in the `catch` block is the single most-forgotten line in concurrent Java code. Catching `InterruptedException` **clears** the thread's interrupted status; if you don't restore it, code further up the call stack that also checks for interruption (to know it should stop early) will never find out the thread was ever interrupted at all.

This exact block appears, essentially verbatim, in `AuditTrail.shutdown()`, `BatchReportService.shutdown()`, `ScheduledGpaRecalculationJob.stop()`, and `StatisticsDashboard.shutdownExecutor()` — four independent classes, one idiom, applied consistently.

Design for testability: inject the executor

```
// Production constructor — real behavior, real threads public
BatchReportService(StudentManager sm, ReportGenerator rg, FileExporter fe, int
threadCount) { this(sm, rg, fe, threadCount, () ->
Executors.newFixedThreadPool(threadCount)); } // Test-only constructor — a
Supplier<ExecutorService> lets a test hand in a mock public
BatchReportService(StudentManager sm, ReportGenerator rg, FileExporter fe, int
threadCount, Supplier<ExecutorService> executorFactory) { ... }
```

> ***Used in this project:*** *`BatchReportService`, `AuditTrail` (`withExecutor(...)`), `ScheduledGpaRecalculationJob` (`start(ScheduledExecutorService)`), and `StatisticsDashboard` (`start(ThreadPoolExecutor)`)* ***all*** *expose a way to substitute a mocked executor.* ***Why:*** *verifying real concurrent timing (a 30-second shutdown timeout, an interrupt during `awaitTermination`, a scheduled task that fires "daily") is either impossibly slow or genuinely flaky if a test has to wait on the real clock/real threads. Injecting a mock executor lets a test assert "my code called `shutdownNow()` after `awaitTermination` returned false" as a plain, fast, deterministic interaction-verification test (Mockito) instead of an actual 30-second sleep.*

Other practices worth internalizing

- **Prefer high-level utilities over hand-rolled `wait()`/`notify()`.** `ExecutorService`, `BlockingQueue`, `CountDownLatch`, and `ConcurrentHashMap` have all been reviewed, battle-tested, and optimized far beyond what hand-written synchronization is likely to achieve correctly on the first (or fifth) try.
- **Minimize the scope of synchronized/locked code.** Lock only what actually needs protecting — holding a lock across an I/O call (a file write, a network request) turns a brief critical section into a throughput bottleneck for every other thread waiting on that lock.
- **Prefer immutable objects for shared state.** An immutable object (like this project's `GradeRecord`/`StudentRecord`, or the `record` types used throughout `main.concurrent`, e.g. `AuditEntry`, `StudentReportOutcome`, `BatchResult`, `DashboardSnapshot`) can never be caught mid-mutation by another thread — there's no race condition to have if nothing ever changes after construction.

- **Never call `Thread.stop()`, `Thread.suspend()`, or `Thread.resume()`.** All three are deprecated for correctness reasons — `stop()` in particular can release locks mid-critical-section, corrupting shared state. Use a cooperative flag (`volatile boolean running`, `AtomicBoolean`, or `Thread.interrupt()` + checking `isInterrupted()`) instead — exactly the pattern `StatisticsDashboard`/`ScheduledGpaRecalculationJob` use.
- **Testing concurrent code:** favor tests that inject a controllable executor (above) or that assert on the *outcome* (e.g. "after `generateBatch()` returns, all N files exist") over tests that try to catch a race in the act by sleeping and hoping — timing-dependent "did the race happen" tests are close to the definition of a flaky test.

---

14. Cheat Sheet

Need	Reach for	Not	Why
Run something in the background once	`ExecutorService.submit(Runnable/Callable)`	raw `new Thread(...).start()`	Reuses pooled threads, gives you a `Future`, has a lifecycle you control
Serialize writes so they never interleave	`Executors.newSingleThreadExecutor`	manual `synchronized` around every write site	Structural guarantee instead of discipline
Bounded, CPU/IO-bound parallel work	`Executors.newFixedThreadPool(n)`	`parallelStream()` on a tiny collection	Predictable pool size, per-task failure isolation, measurable
Bursty, short-lived, occasional tasks	`Executors.newCachedThreadPool()`	a fixed pool sized for peak load	No idle-thread cost between bursts
A recurring/scheduled job	`ScheduledExecutorService.scheduleAtFixedRate`	`while(true) { sleep(); ... }` loop you hand-roll	Correct interval handling, cancellable, one clear API
Return a value from a background task	`Callable<T>` + `Future<T>`	`Runnable` with a shared mutable "result" field	Type-safe, exception-safe, no extra synchronization needed
Chain async steps without blocking	`CompletableFuture`	blocking `Future.get()` chains	Non-blocking composition, error handling built in
A flag one thread sets and another polls	`volatile boolean`	plain `boolean`	Guarantees visibility across threads
A counter under contention	`AtomicInteger`/`AtomicLong`	`synchronized` + `int`	Lock-free, faster under contention
A map read far more than it's written	`ConcurrentHashMap`	`synchronized HashMap`	Concurrent reads never block each other

A list read far more than it's written	`CopyOnWriteArrayList`	`synchronized ArrayList`	Lock-free reads/iteration
Reads vastly outnumber writes, need real consistency	`ReentrantReadWriteLock`	plain `ReentrantLock`/`synchronized`	Readers never block other readers
Only one of two racing threads should "win"	`AtomicBoolean.compareAnd`	a `synchronized` flag check-then-set	Atomic, lock-free, one clear winner
Divide-and-conquer over a big in-memory collection	Fork/Join / `parallelStream()`	manual thread splitting	Work-stealing keeps all cores busy automatically
Unbounded event stream with backpressure	Reactive (`Flow`, Reactor, RxJava)	polling / unbounded queues	Consumer can signal "slow down" to the producer

---

15. Where This Appears In This Codebase

File	Concurrency tools used	Concept sections
`main/concurrent/AuditTrail.jav`	`Executors.newSingleThreadExecutor`, daemon `Thread`, `Future`, `CompletableFuture.completedFuture`, `CopyOnWriteArrayList`	§3, §5, §6, §9
`main/concurrent/BatchReportSer`	`Executors.newFixedThreadPool`, `Callable`-shaped lambdas, `Future`, `ExecutionException` handling, `Supplier<ExecutorService>` for testability	§5, §12, §13
`main/concurrent/ScheduledGpaRe`	`ScheduledExecutorService`, `scheduleAtFixedRate`, daemon `Thread`, `volatile` publish-once fields	§3, §4, §5, §12
`main/concurrent/StatisticsDashboard.java`	Manual-joker `Thread` + `join(timeout)`, hand-built `ThreadPoolExecutor` (cached-pool shape), `AtomicBoolean.compareAndSet`, `SynchronousQueue`	§3, §5, §6, §7
`main/concurrent/LruCache.java`	`ConcurrentHashMap`, `AtomicLong` (clock + hit/miss counters)	§6, §7

`main/manager/GradeManager.java`	`ReentrantReadWriteLock` (`readLocked`, `addGrade`)	§7
(not present anywhere in this codebase)	Fork/Join, parallel streams, `CountDownLatch`, `Semaphore`, reactive streams, deadlock/livelock scenarios	§8, §10, §11

---

16. Study Plan — What to Learn Next

1. **Read `GradeManager`, then all five files in `main/concurrent/`, in the order listed in the table above** — each one introduces exactly one or two new tools on top of the last, in roughly increasing sophistication.
2. **Run the tests for each class** (`mvn test -Dtest=tests.concurrent.*`) and read the *test* files, not just the production code — `ScheduledGpaRecalculationJobTest`/`StatisticsDashboardTest`/`BatchReportServiceMockitoTe` show exactly how the injectable-executor pattern (§13) turns "wait 30 real seconds" into a fast, deterministic assertion.
3. **Deliberately break something** in a scratch file: remove a `finally` block around `unlock()`, remove `Thread.currentThread().interrupt()` from a `catch (InterruptedException e)`, or make `LruCache` use a plain `HashMap` instead of `ConcurrentHashMap` — then try to write a test that reliably demonstrates the bug. This is the fastest way to build real intuition for *why* each safeguard in this guide exists.
4. **Once comfortable with everything in §1–§7 and §12–§13** (the parts actually used here), branch out to Fork/Join (§8) and `CompletableFuture` chaining (§9) with a toy project — a parallel word-count or a small async pipeline are the classic next steps.
5. **Treat §10 (reactive) and §11 (deadlock/livelock) as "recognize it, know the fix" knowledge** rather than something to practice in isolation — they matter most when reviewing *other* people's code or diagnosing a production incident, not as something you'll typically reach for first when writing new code in a codebase like this one.